453. Minimum Moves to Equal Array Elements

Daily Question - November 1, 2026

Problem Statement

Given an integer array nums of size n, return the minimum number of moves required to make all array elements equal.

In one move, you can increment n - 1 elements of the array by 1.

Examples

Example 1: nums = [1,2,3]. Output: 3. Explanation: Each move increments two elements. [1,2,3] -> [2,3,3] -> [3,4,3] -> [4,4,4].

Example 2: nums = [1,1,1]. Output: 0. All elements are already equal.

Constraints

n == nums.length; 1 <= n <= 100000; -1000000000 <= nums[i] <= 1000000000. The answer is guaranteed to fit in a 32-bit integer.

Key Insight

Incrementing n - 1 elements by 1 is equivalent, in terms of relative differences, to decrementing one element by 1. Thus, the task reduces to finding how many conceptual decrements are needed to bring every element down to the minimum value.

Let minimum = min(nums). For each element, the required conceptual decrements are nums[i] - minimum. Therefore, the answer is the sum of these differences:

answer = sum(nums) - minimum * n

This is optimal because each move can reduce at most one element relative to the others, and every element above the minimum must be reduced by exactly its difference from the minimum.

Complete Java Solution

```
class Solution {
    public int minMoves(int[] nums) {
        long sum = 0;
        int minimum = Integer.MAX_VALUE;

        for (int value : nums) {
            sum += value;
            minimum = Math.min(minimum, value);
        }

        return (int) (sum - (long) minimum * nums.length);
    }
}
```

Correctness

A move increases n - 1 elements. Subtracting 1 from every element leaves all pairwise differences unchanged, so such a move is equivalent to decreasing one chosen element by 1. To make all elements equal, every element must be brought to the same final value. The minimum possible final value is the original minimum, because no element can be decreased below its original value under the equivalent operation. Hence each element nums[i] requires exactly nums[i] - minimum conceptual decrements, and their sum is the minimum number of moves.

Complexity

Time: O(n), since the array is scanned once. Auxiliary space: O(1), because only scalar variables are used.

Implementation Notes

Use long for sum and for minimum * n to avoid intermediate integer overflow. The final result is guaranteed to fit in an int, so it is safe to cast at the end. If all values are already equal, the formula naturally returns 0.